周报2

本周已刷题目

力扣：简单 $\times 9$ 中等 $\times 5$

洛谷：普及 $\times 1$

牛客：$\times 1$

- 数组
  1. P2615NOIP 2015 提高组 神奇的幻方（普及）
  2. 1877. 数组中最大数对和的最小值 (**)
  3. 1984. 学生分数的最小差值 (*)
- 链表
  1. 86. 分隔链表 (**)
- 栈和队列
  1. 622. 设计循环队列 (**)
  2. 232. 用栈实现队列 (*)
  3. 225. 用队列实现栈 (*)
  4. 155. 最小栈 (**)
  5. 641. 设计循环双端队列 (**)
- 二叉树
  1. 144. 二叉树的前序遍历 （递归）(*)

     144. 二叉树的前序遍历 （迭代）
  2. 94. 二叉树的中序遍历 （递归）(*)

     94. 二叉树的中序遍历 （迭代）
  3. 145. 二叉树的后序遍历 （递归）(*)

     145. 二叉树的后序遍历 （迭代）
- 排序
  1. 排序（归并排序）_牛客题霸_牛客网 (**)

学习知识点

1. c语言中`a[i]`的底层执行过程就是`*(a + i)`，写成`i[a]`的效果其实是一样的，但阅读性差，实际没人使用这种写法。
2. 队列是一种先进先出的数据结构，栈是先进后出的数据结构，可以通过数组来实现，栈和队列可以相互实现。
3. c语言`<stdlib.h>`头文件中有`malloc()`申请连续空间，`calloc()`申请连续空间并初始化，`realloc()`扩容空间，`free()`释放内存，这些函数可用于c语言动态内存分配和释放内存。分配的内存位于堆，使用完成后要用`free()`释放指针指向的内存，避免内存泄漏。
4. 当同时在队列中元素不超过k个时，可以用长度为k的数组实现队列，即循环队列。

5. c语言可以定义常数，常数存储在常量区，只读，程序执行时不能修改

```
#define MAXSIZE 8001
```

6. 二叉树
   - 二叉树的节点
     - 每个节点有左右子节点
   - 二叉树的遍历
     - 先序：任何一颗树满足先头，再左，再右的遍历方式
     - 中序：先左，再头，再右
     - 后序：先左，再右，后中

7. 递归序

```
public class BinaryTreeTraversalRecursion {

    public static class TreeNode {
        public int val;
        public TreeNode left;
        public TreeNode right;

        public TreeNode(int v) {
            val = v;
        }
    }

    // 递归基本样子，用来理解递归序
    public static void f(TreeNode head) {
        if (head == null) {
            return;
        }
        // 第1次到head
        f(head.left);
        // 第2次到head
        f(head.right);
        // 第3次到head
    }
}
```

8. 用栈实现非递归遍历二叉树
   - 先序：

     核心循环：

     1. 出栈时处理节点
     2. 先把右孩子压入栈，再压入左孩子，这样出栈时先出左，再出右
   - 中序：

     核心循环：

     1. 先把整条左支压入栈

2. 直到`root`指向空后，让`root`重新指向并处理上一节点
3. 处理过的节点出栈，保证不会重复处理同一个节点
4. 处理一个节点后让`root`指向该节点右子树，把右子树左支也压入栈

- 后序：

  核心循环：

  1. 哨兵节点`root`指向上一次处理的节点
  2. 栈顶的节点为待处理的当前节点
  3. 检查当前节点`cur`是否有为处理过的左右子树，若有则压入站内，若没有则处理当前节点，将当前节点写入`result`
  4. 让`root`指向已处理的当前节点，并从栈内弹出

9. c++中取消同步流，提高IO效率

```
std::ios::sync_with_stdio(false);
std::cin.tie(nullptr);
```

10. 递归
    - 系统栈：
      - 递归的调用过程依赖系统栈
      - 系统栈是独立于内存的空间
    - 所有递归都能改成迭代
      - 迭代是通过自己在内存建立栈实现的
      - 工程上几乎都要改，除非确定数据量不大，或递归层数不深。例如：归并排序、快速排序、线段树、平衡树。

刷题笔记

- 数组
  1. P2615NOIP 2015 提高组 神奇的幻方
     - 用`row`和`column`直接记录上一个填入的数的位置
     - 每次填入新的数修改`row`和`column`的值
- 链表
  1. 86. 分隔链表
     - 定义4个指针分别记录左右链表头尾，定义`next`指针记录当前节点下一个，每轮循环完成后`head`跳转到next
     - 处理当前节点前先将当前节点断连，避免最后链表成环
     - 最后将左右链表连起来，连之前为防止`ltl`为`NULL`报错，先检查左链表，为空则直接返回右链表。
- 队列
  1. 622. 设计循环队列

     用数组实现循环队列

     - 用`size`和`limit`控制`head`和`tail`指针归位
     - 创建循环队列时用`malloc()`为结构体指针和数组指针分配内存

       `myCircularQueueCreate`函数中加入空指针保护

```
if (cirque == NULL) return NULL;
```

- 用`tail+1`对`limit`取模控制`tail`归0，`head`同理

2. 225. 用队列实现栈
   - 创建循环队列
   - 核心步骤：元素加入队列后，若前面已有n个数，则依次将他们从队列头取出，并从队尾重新加入，执行n次。
3. 155. 最小栈
   - 设置两个栈，一个存储原数据，一个最小栈，两个栈内数据量一样，维护`size`即可获取经过`pop`、`push`的栈顶元素和栈内最小元素，即最小栈顶元素
4. 641. 设计循环双端队列
   - 和循环队列一样通过数组实现
   - `l`和`r`指针移动时判断是否越界

```
// 头部添加，l左移
l = (l == 0) ? limit - 1 : l - 1;
// 尾部添加，r右移
r = (r + 1) % limit;
/* 把(r + 1)对limit取模，
 * 若limit为5，数组索引最多到4，
 * (4 + 1) % 5 == 0
 * 取模可以控制r索引不越界
*/
// 头部删除，l右移
l = (l + 1) % limit;
// 尾部删除，r左移
r = (r == 0) ? limit - 1 : r - 1;
```

   - 添加元素时若队列为空，则将`l`和`r`指针归0，并在索引0处添加新元素

- 排序
  1. 排序（归并排序）_牛客题霸_牛客网

     用递归实现，时间复杂度$O(n*logN)$
     - 分治：

       > 先二分处理，直到分为基本单元，即长度为一的数组，天然有序

     - 归并

       > 合并两个有序数组
       >
       > 先检查边界，判断下标是否越界

下周计划

学习`c++`的基础语法，学习归并分治思想并练习相应题目，学习一些排序算法

代码

1.数组

1. P2615NOIP 2015 提高组 神奇的幻方

```cpp
#include <iostream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int matrix[39][39];
    int n;
    std::cin >> n;
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            matrix[i][j] = 0;
        }
    }
    int row, column;
    int npow = n * n;

    row = 0;
    column = n / 2;
    matrix[0][column] = 1;
    // 填数循环
    for (i = 2; i <= npow; i++) {
        if (row == 0) {
            if (column == n - 1) {
                // i - 1 在第一行最后一列
                matrix[++row][column] = i;
            } else {
                // i - 1 在第一行，不在最后一列
                row = n - 1;
                matrix[row][++column] = i;
            }
        } else {
            if (column == n - 1) {
                // i - 1 不在第一行，在最后一列
                column = 0;
                matrix[--row ][0] = i;
            } else {
                // i - 1 不在第一行，不在最后一列
                if (column + 1 < n && matrix[row - 1][column + 1] == 0) {
                    // i - 1右上方未填入
                    matrix[--row][++column] = i;
                } else {
                    matrix[++row][column] = i;
                }
```

```
            }
        }
    }

    // 遍历
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            std::cout << matrix[i][j] << ' ';
        }
        std::cout << "\n";
    }
}
```

2. 1877. 数组中最大数对和的最小值

```
class Solution {
public:
    int minPairSum(vector<int>& nums) {
        int h = 0;
        int cur;
        std::sort(nums.begin(), nums.end());
        for (int i = 0; i < nums.size() / 2; i++) {
            // 排序后首尾相加
            cur = nums[i] + nums[nums.size() - 1 - i];
            if (cur > h) {
                h = cur;
            }
        }
        return h;
    }
};
```

3. 1984. 学生分数的最小差值

```
class Solution {
public:
    int minimumDifference(vector<int>& nums, int k) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        int result = 10e5;
        for (int i = 0; i + k - 1 < n; i++) {
            if (nums[i + k - 1] - nums[i] < result) {
                result = nums[i + k - 1] - nums[i];
            }
        }
        return result;
    }
};
```

2.链表

1. 86. 分隔链表

```c
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */
struct ListNode* partition(struct ListNode* head, int x) {
    struct ListNode* lhd = NULL;
    struct ListNode* ltl = NULL;
    struct ListNode* rhd = NULL;
    struct ListNode* rtl = NULL;
    struct ListNode* next = NULL;

    while(head != NULL)
    {
        next = head->next; // 记录下一节点
        head->next = NULL; // 断连节点，防止形成环
        if(head->val >= x)
        {
            // 右链表头为空，则头设为当前节点
            if(rhd == NULL)
            {
                rhd = head;
            }
            else
            {
                // 右链表尾部接当前节点
                rtl->next = head;
            }
            rtl = head;
        }
        else
        {
            if(lhd == NULL)
            {
                lhd = head;
            }
            else
            {
                ltl->next = head;
            }
            ltl = head;
        }
        head = next; // 跳转下一节点
    }

    if(lhd == NULL)
    {
        return rhd;
    }
```

```
    ltl->next = rhd; // 连接左右链表
    return lhd;
}
```

3.栈和队列

1. 622. 设计循环队列

```
/*
 * 当同时在队列中元素不超过k个时，可以用长度为k的数组实现队列
 * 当head,tail指针越界时，回到0索引位置
*/


typedef struct {
    int* queue;
    int head, tail, size, limit;
} MyCircularQueue;

bool myCircularQueueIsEmpty(MyCircularQueue* obj);
bool myCircularQueueIsFull(MyCircularQueue* obj);


// 创建循环队列
MyCircularQueue* myCircularQueueCreate(int k) {
    MyCircularQueue* cirque = (MyCircularQueue*)malloc(sizeof(MyCircularQueue));
    if (cirque == NULL) return NULL;
    cirque->queue = malloc(k * sizeof(int));
    cirque->limit = k;
    cirque->head = cirque->tail = cirque->size = 0;
    return cirque;
}

// 添加元素,队列满返回false且不添加,队列不满返回true且添加
bool myCircularQueueEnQueue(MyCircularQueue* obj, int value) {
    if (obj == NULL) return false;
    if(myCircularQueueIsFull(obj)) return false;
    else
    {
        obj->queue[obj->tail] = value;
        obj->tail = (obj->tail + 1) % obj->limit; // 若指针过尾则归0
        obj->size++;
        return true;
    }
}

// 弹出头部元素，若为空返回false，否则弹出头部且返回true
bool myCircularQueueDeQueue(MyCircularQueue* obj) {
    if (obj == NULL) return false;
    if(myCircularQueueIsEmpty(obj)) return false;
    else
```

```
    {
        obj->head = (obj->head + 1) % obj->limit; // 若指针过尾则归0
        obj->size--;
        return true;
    }
}

// 返回头部元素
int myCircularQueueFront(MyCircularQueue* obj) {
    if (obj == NULL) return -1;
    if(myCircularQueueIsEmpty(obj)) return -1;
    else
    {
        return obj->queue[obj->head];
    }
}

// 返回尾部元素
int myCircularQueueRear(MyCircularQueue* obj) {
    if (obj == NULL) return -1;
    if(myCircularQueueIsEmpty(obj)) return -1;
    else
    {
        int last = obj->tail == 0 ? (obj->limit - 1) : obj->tail - 1;
        return obj->queue[last];
    }
}

bool myCircularQueueIsEmpty(MyCircularQueue* obj) {
    if (obj == NULL) return false;
    return obj->size == 0;
}

bool myCircularQueueIsFull(MyCircularQueue* obj) {
    if (obj == NULL) return false;
    return obj->size == obj->limit;
}

void myCircularQueueFree(MyCircularQueue* obj) {
    if (obj == NULL) return;
    free(obj->queue);
    free(obj);
}

/**
 * Your MyCircularQueue struct will be instantiated and called as such:
 * MyCircularQueue* obj = myCircularQueueCreate(k);
 * bool param_1 = myCircularQueueEnQueue(obj, value);

 * bool param_2 = myCircularQueueDeQueue(obj);

 * int param_3 = myCircularQueueFront(obj);

```

```
 * int param_4 = myCircularQueueRear(obj);

 * bool param_5 = myCircularQueueIsEmpty(obj);

 * bool param_6 = myCircularQueueIsFull(obj);

 * myCircularQueueFree(obj);
*/
```

2. 232. 用栈实现队列

```
/*
* 用两个栈实现队列
*/


typedef struct {
    int* in;
    int* out;
    int intop, outtop;
} MyQueue;

// 判断out栈是否为空
bool out_empty(MyQueue* q)
{
    return (q->outtop == 0);
}

void inToOut(MyQueue* q)
{
    if(out_empty(q)) // out栈为空才倒数据
    {
        while(q->intop != 0) // in栈不为空时循环
        {
            q->out[q->outtop++] = q->in[--q->intop]; // 倒数据
        }
    }
}

// 创建队列
MyQueue* myQueueCreate() {
    MyQueue* q = (MyQueue*)malloc(sizeof(MyQueue));
    q->in = (int*)malloc(100 * sizeof(int)); // in栈
    q->out = (int*)malloc(100 * sizeof(int)); // out栈
    q->intop = q->outtop = 0;
    return q;
}

// 向队列添加元素
void myQueuePush(MyQueue* obj, int x) {
    obj->in[obj->intop++] = x;
    inToOut(obj);
}
```

```
43
44  // 从队列中取出元素
45  int myQueuePop(MyQueue* obj) {
46      inToOut(obj);
47      if(out_empty(obj))
48          return 0;
49      return obj->out[--obj->outtop];
50  }
51
52  // 返回队列头部元素
53  int myQueuePeek(MyQueue* obj) {
54      inToOut(obj);
55      if(out_empty(obj))
56          return 0;
57      else
58          return obj->out[obj->outtop - 1];
59  }
60
61  // 判断队列是否为空
62  bool myQueueEmpty(MyQueue* obj) {
63      return obj->intop == 0 && obj->outtop == 0;
64  }
65
66  void myQueueFree(MyQueue* obj) {
67      free(obj->in);
68      free(obj->out);
69      free(obj);
70  }
71
72  /**
73   * Your MyQueue struct will be instantiated and called as such:
74   * MyQueue* obj = myQueueCreate();
75   * myQueuePush(obj, x);
76
77   * int param_2 = myQueuePop(obj);
78
79   * int param_3 = myQueuePeek(obj);
80
81   * bool param_4 = myQueueEmpty(obj);
82
83   * myQueueFree(obj);
84  */
```

3. 225. 用队列实现栈

```
1  /*
2  用一个队列实现栈
3  */
4
5  // 栈结构体，但是用循环队列实现
6  typedef struct {
7      int* queue;
8      int front, tail, size, limit;
```

```c
} MyStack;

// 创建栈
MyStack* myStackCreate() {
    MyStack* sta = (MyStack*)malloc(sizeof(MyStack));
    (*sta).limit = 100;
    (*sta).queue = (int*)malloc((*sta).limit * sizeof(int));
    (*sta).front = (*sta).tail = (*sta).size = 0;
    return sta;
}

// 循环队列push
void pushQueue(MyStack* obj, int x)
{
    if((*obj).size == (*obj).limit)
    {
        return;
    }
    else
    {
        (*obj).queue[(*obj).tail] = x;
        (*obj).size++;
        (*obj).tail = ((*obj).tail + 1) % (*obj).limit;
    }
}

// 循环队列pop
int popQueue(MyStack* obj)
{
    if((*obj).size == 0)
    {
        return 0;
    }
    else
    {
        int pop = (*obj).queue[(*obj).front];
        (*obj).size--;
        (*obj).front = ((*obj).front + 1) % (*obj).limit;
        return pop;
    }
}

// 入栈,单次操作时间复杂度O(n)
void myStackPush(MyStack* obj, int x) {
    int n = (*obj).size;
    // 入栈,用单个队列实现栈的关键步骤，前面已有n个元素,前n个元素须出队列再重进
    pushQueue(obj, x);
    for(int i = 0; i < n; i++)
    {
        pushQueue(obj, popQueue(obj));
    }
}

```

```c
int myStackPop(MyStack* obj) {
    return popQueue(obj); // 出栈，实际从循环队列中取出元素
}

int myStackTop(MyStack* obj) {
    return (*obj).queue[(*obj).front]; // 栈顶元素，实际是队列头
}

bool myStackEmpty(MyStack* obj) {
    return (*obj).size == 0;
}

void myStackFree(MyStack* obj) {
    free((*obj).queue);
    free(obj);
}

/**
 * Your MyStack struct will be instantiated and called as such:
 * MyStack* obj = myStackCreate();
 * myStackPush(obj, x);

 * int param_2 = myStackPop(obj);

 * int param_3 = myStackTop(obj);

 * bool param_4 = myStackEmpty(obj);

 * myStackFree(obj);
*/
```

4. 155. 最小栈

```c
#define MAXSIZE 8001 // 定义常数，最大同时栈内数据量


typedef struct {
    int data[MAXSIZE]; // 原数据
    int min[MAXSIZE]; // 最小栈
    int size;
} MinStack;

MinStack* minStackCreate() {
    MinStack* ms = (MinStack*)malloc(sizeof(MinStack));
    ms->size = 0;
    return ms;
}

void minStackPush(MinStack* obj, int val) {
    if(obj->size < MAXSIZE)
        obj->data[obj->size] = val;
    else return;
    // 最小栈
```

```
    if(obj->size == 0 || val <= (*obj).min[obj->size - 1])
    {
        (*obj).min[obj->size] = val;
    }
    else
    {
        // 若新数据比原来最小栈顶大，则最小栈再次压入栈顶元素
        (*obj).min[obj->size] = (*obj).min[obj->size - 1];
    }
    obj->size++;
}

void minStackPop(MinStack* obj) { // 删除栈顶元素
    if(obj->size == 0) return;
    else
    {
        obj->size--;
    }
}

int minStackTop(MinStack* obj) { // 返回原栈顶元素
    if(obj->size == 0) return 0;
    return (*obj).data[obj->size - 1];
}

int minStackGetMin(MinStack* obj) { // 获取栈内最小元素，即最小栈顶元素
    if(obj->size == 0) return 0;
    return (*obj).min[obj->size - 1];
}

void minStackFree(MinStack* obj) {
    free(obj);
}

/**
 * Your MinStack struct will be instantiated and called as such:
 * MinStack* obj = minStackCreate();
 * minStackPush(obj, val);

 * minStackPop(obj);

 * int param_3 = minStackTop(obj);

 * int param_4 = minStackGetMin(obj);

 * minStackFree(obj);
*/
```

5. 641. 设计循环双端队列

```
class MyCircularDeque {
    int* deque;
    int l, r, size, limit;
```

```
4
5   public:
6       MyCircularDeque(int k) {
7          deque = (int*)malloc(k * sizeof(int));
8          limit = k;
9          l = r = size = 0;
10      }
11
12      bool insertFront(int value) {
13          if(isFull()) {
14              return false;
15          } else if(isEmpty()) {
16              // 为空直接在索引0存
17              l = r = 0;
18              deque[0] = value;
19          } else {
20              // 非空l指针左移再存储
21              l = (l == 0) ? limit - 1 : l - 1;
22              deque[l] = value;
23          }
24          size++;
25          return true;
26      }
27
28      bool insertLast(int value) {
29          if(isFull()) {
30              return false;
31          } else if(isEmpty()) {
32              // 为空直接在索引0存
33              l = r = 0;
34              deque[0] = value;
35          } else {
36              // 非空r指针右移再存储
37              r = (r + 1) % limit;
38              deque[r] = value;
39          }
40          size++;
41          return true;
42      }
43
44      bool deleteFront() {
45          if(isEmpty()) {
46              return false;
47          } else {
48              // 右移l指针
49              l = (l + 1) % limit;
50              size--;
51              return true;
52          }
53      }
54
55      bool deleteLast() {
56          if(isEmpty()) {
```

```
            return false;
        } else {
            // 左移r指针
            r = (r == 0) ? limit - 1 : r - 1;
            size--;
            return true;
        }
    }

    int getFront() {
        if(isEmpty()) {
            return -1;
        } else {
            return deque[l];
        }
    }

    int getRear() {
        if(isEmpty()) {
            return -1;
        } else {
            return deque[r];
        }
    }

    bool isEmpty() {
        return size == 0;
    }

    bool isFull() {
        return size == limit;
    }

    void freeDeque() {
        free(deque);
    }
};

/**
 * Your MyCircularDeque object will be instantiated and called as such:
 * MyCircularDeque* obj = new MyCircularDeque(k);
 * bool param_1 = obj->insertFront(value);
 * bool param_2 = obj->insertLast(value);
 * bool param_3 = obj->deleteFront();
 * bool param_4 = obj->deleteLast();
 * int param_5 = obj->getFront();
 * int param_6 = obj->getRear();
 * bool param_7 = obj->isEmpty();
 * bool param_8 = obj->isFull();
 */
```

4.二叉树

1. 144. 二叉树的前序遍历
    - 递归实现

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> result;
        preorder(root, result);
        return result;
    }

private:
    void preorder(TreeNode* root, vector<int> &result) {
        if (root == nullptr) {
            return;
        }
        result.push_back(root->val); // 输出当前节点
        preorder(root->left, result); // 遍历左子树
        preorder(root->right, result); // 遍历右子树
    }
};
```

    - 迭代实现（基于栈的非递归实现）

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
```

```
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> result;
        if (root == nullptr) {
            return result;
        }
        stack<TreeNode*> temp;

        temp.push(root); // 将头节点入栈
        while (!temp.empty()) {
            root = temp.top();
            temp.pop();
            result.push_back(root->val); // 处理当前节点

            // 先压入右节点，再压入左节点，出栈时左先出
            if (root->right != nullptr) {
                temp.push(root->right);
            }

            if (root->left != nullptr) {
                temp.push(root->left);
            }
        }
        return result;
    }
};
```

2. 94. 二叉树的中序遍历
    - 递归

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> result;
        if (root == nullptr) {
            return result;
        }

        inorder(root, result);
        return result;
    }
```

```
23
24      void inorder(TreeNode* root, vector<int> &result) {
25          if (root == nullptr) {
26              return;
27          }
28
29          inorder(root->left, result);
30          result.push_back(root->val);
31          inorder(root->right, result);
32      }
33  };
```

- 迭代

```
1   /**
2    * Definition for a binary tree node.
3    * struct TreeNode {
4    *     int val;
5    *     TreeNode *left;
6    *     TreeNode *right;
7    *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
8    *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
9    *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
     right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      vector<int> inorderTraversal(TreeNode* root) {
15          vector<int> result;
16          if (root == nullptr) {
17              return result;
18          }
19
20          iterate_inorder(root, result);
21          return result;
22      }
23
24      void iterate_inorder(TreeNode* root, vector<int> &result) {
25          if (root != nullptr) {
26              stack<TreeNode*> stk;
27              while (!stk.empty() || root != nullptr) {
28                  // 初始时栈为空，但节点不为空；处理完节点后栈重新变为空
29                  // 把整条左支压入栈
30                  if (root != nullptr) {
31                      stk.push(root);
32                      root = root->left;
33                  } else {
34                      // 左支压完后root回到上一层节点，并处理节点
35                      root = stk.top();
36                      stk.pop(); // 当前节点处理后取出
37                      result.push_back(root->val);
38                      // root指向右节点,将当前节点右子树左支压入栈
```

```
                    root = root->right;
                }
            }
        }
    }
};
```

3. 145. 二叉树的后序遍历
    - 递归

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> result;
        if (root == nullptr) {
            return result;
        }

        posorder(root, result);
        return result;
    }

    void posorder(TreeNode* root, vector<int> &result) {
        if (root == nullptr) {
            return; // 节点为空回溯
        }

        posorder(root->left, result); // 递归左子树
        posorder(root->right, result); // 右子树
        result.push_back(root->val); // 当前节点值写入结果
    }
};
```

    - 迭代

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
```

```
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> result;
        postorder(root, result);
        return result;
    }

    void postorder(TreeNode* root, vector<int> &result) {
        if (root == nullptr) {
            return;
        }
        stack<TreeNode*> nodeStack;
        nodeStack.push(root);

        // 若始终没打印过节点，root就是头节点
        // 否则root变为上次打印的节点
        while (!nodeStack.empty()) {
            TreeNode* cur = nodeStack.top();
            if (cur->left != nullptr && root != cur->left && root != cur->right) {
                // 有左树且左树没处理过
                nodeStack.push(cur->left);
            } else if (cur->right != nullptr && root != cur->right) {
                // 有右树且右树没处理过
                nodeStack.push(cur->right);
            } else {
                // 没有左右子树或左右子树处理过
                result.push_back(cur->val);
                root = cur;
                nodeStack.pop();
            }
        }
    }
};
```

5.排序

1. 排序（归并排序）_牛客题霸_牛客网

```
#include <iostream>
#include <array>
using namespace std;
```

```cpp
enum {MAXN = 501};

void mergeSort(array<int, MAXN>& arr, array<int, MAXN>& assistant, int l, int r);
void merge(array<int, MAXN>& arr, array<int, MAXN>& assistant, int l, int m, int r);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    array<int, MAXN> arr {0};
    array<int, MAXN> assistant {0};
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    mergeSort(arr, assistant, 0, n - 1);
    for (int i = 0; i < n; i++) {
        cout << arr[i] << ' ';
    }
}

void mergeSort(array<int, MAXN>& arr, array<int, MAXN>& assistant, int l, int r) {
    if (l == r) {
        return;
    }
    int m = (l + r) / 2;
    mergeSort(arr, assistant, l, m);
    mergeSort(arr, assistant, m + 1, r);
    merge(arr, assistant, l, m, r);
}

void merge(array<int, MAXN>& arr, array<int, MAXN>& assistant, int l, int m, int r) {
    int i = l;
    int a = l;
    int b = m + 1;
    while (a <= m && b <= r) {
        // a和 b未越界
        assistant[i++] = arr[a] <= arr[b] ? arr[a++] : arr[b++];
    }
    while (a <= m) {
        // a 未越界
        assistant[i++] = arr[a++];
    }
    while (b <= r) {
        // b 未越界
        assistant[i++] = arr[b++];
    }
    for (i = l; i <= r; i++) {
        arr[i] = assistant[i];
```

```
    }
}
// 64 位输出请用 printf("%lld")
```